Name:- Raghav Anil Bhise
Roll No. :- 391061
PRN:- 22311792

1. Title

Bag of Words Implementation Using Python (From Scratch)

---

2. Aim

To implement the **Bag of Words (BoW) model from scratch using Python** for converting textual data into numerical vectors for Natural Language Processing.

---

3. Objectives

- To understand the working of the Bag of Words model
- To perform basic **text preprocessing** such as lowercasing, tokenization, and punctuation removal
- To build a **vocabulary** manually from the given corpus
- To compute **word frequency vectors** for each document

---

4. Theory

4.1 Natural Language Processing

Natural Language Processing (NLP) enables computers to process and analyze human language. Since text data cannot be directly used by machine learning algorithms, it must be transformed into numerical form.

4.2 Bag of Words (BoW) Model

The Bag of Words model is a simple text representation technique where:

- Each document is treated as a **collection of words**
- Grammar and word order are ignored
- Only **word frequency** is considered

4.3 Steps in Bag of Words (As per the File)

Step 1: Text Preprocessing

- Convert text to lowercase
- Remove punctuation
- Tokenize text into words

Step 2: Vocabulary Construction

- Extract all unique words from the corpus
- Store them as a vocabulary list

Step 3: Vectorization

- Count the occurrence of each vocabulary word in every document
- Represent each document as a numerical vector

4.4 Advantages and Limitations

Advantages

- Simple and easy to implement
- Useful for basic NLP tasks

Limitations

- Ignores semantics and word order
- High dimensionality for large vocabularies

---

5. Program Codes

Bag of Words Implementation Using Python (From Scratch)

Step 1: Preprocessing the Text Data

Defining a simple function to process text, including tokenization, lowercasing, and removing punctuation.

```python
from collections import defaultdict
import string

# Sample text data: sentences
corpus = [
    "Python is easy to learn and enjoyable.",
    "Beyond being enjoyable, Python is extremely versatile.",
    "Practicing Python makes coding more exciting.!",
]
# Function to preprocess text
def preprocess(text):
    # Convert to lowercase
    text = text.lower()
    # Remove punctuation
    text = text.translate(str.maketrans("", "", string.punctuation))
    # Tokenize: split the text into words
    tokens = text.split()
    return tokens

# Apply preprocessing to the sample corpus
processed_corpus = [preprocess(sentence) for sentence in corpus]
print(processed_corpus)
```

```
[['python', 'is', 'easy', 'to', 'learn', 'and', 'enjoyable'], ['beyond', 'being', 'enjoyable', 'python', 'is', 'extremely', 'versatile'], ['practicing', 'python',
```

Step 2: Build Vocabulary

Now, we need to scan through all the documents and build a complete list of unique words, that is our vocabulary.

```python
# Initialize an empty set for the vocabulary
vocabulary = set()

# Build the vocabulary
for sentence in processed_corpus:
    vocabulary.update(sentence)

# Convert to a sorted list
vocabulary = sorted(list(vocabulary))
print("Vocabulary:", vocabulary)
```

```
Vocabulary: ['and', 'being', 'beyond', 'coding', 'easy', 'enjoyable', 'exciting', 'extremely', 'is', 'learn', 'makes', 'more', 'practicing', 'python', 'to', 'vers
```

Step 3: Calculate Word Frequencies and Vectorize

We'll now calculate the frequency of each word in the vocabulary for every document in the processed corpus.

```
def create_bow_vector(sentence, vocab):
    vector = [0] * len(vocab)  # Initialize a vector of zeros
    for word in sentence:
        if word in vocab:
            idx = vocab.index(word)  # Find the index of the word in the vocabulary
            vector[idx] += 1  # Increment the count at that index
    return vector
```

```
#At this point, you will have created a Bag of Words representation for each document in your corpus.


# Create BoW vector for each sentence in the processed corpus
bow_vectors = [create_bow_vector(sentence, vocabulary) for sentence in processed_corpus]
print("Bag of Words Vectors:")
for vector in bow_vectors:
    print(vector)
```

```
Bag of Words Vectors:
[1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0]
[0, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 1, 0, 1]
[0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 0]
```

```
from sklearn.feature_extraction.text import CountVectorizer
# Original corpus
corpus = [
   "Python is easy to learn and enjoyable.",
    "Beyond being enjoyable, Python is extremely versatile.",
    "Practicing Python makes coding more exciting.!",
]
# Create a CountVectorizer Object
vectorizer = CountVectorizer()
# Fit and transform the corpus
X = vectorizer.fit_transform(corpus)
# Print the generated vocabulary
print("Vocabulary:", vectorizer.get_feature_names_out())
# Print the Bag-of-Words matrix
print("BoW Representation:")
print(X.toarray())
total_vocab_count = len(vectorizer.get_feature_names_out())
print("Total vocabulary count:", total_vocab_count)
```

```
Vocabulary: ['and' 'being' 'beyond' 'coding' 'easy' 'enjoyable' 'exciting' 'extremely'
 'is' 'learn' 'makes' 'more' 'practicing' 'python' 'to' 'versatile']
BoW Representation:
[[1 0 0 0 1 1 0 0 1 1 0 0 0 1 1 0]
 [0 1 1 0 0 1 0 1 1 0 0 0 0 1 0 1]
 [0 0 0 1 0 0 1 0 0 0 1 1 1 1 0 0]]
Total vocabulary count: 16
```

Example: Applying Bag of Words

Here are the steps that we will take:

CountVectorizer tokenizes the text, removes punctuation, and lowercase the words automatically.

.fit_transform(corpus) converts the corpus into a document-term matrix, where each row represents a document and each column represents a word from the vocabulary.

X_dense is the dense matrix that represents the frequency of each word in each document.

```
from sklearn.feature_extraction.text import CountVectorizer
# Sample corpus of movie reviews
corpus = [
    "I really like this phone, it works perfectly!",
    "The phone is fine, but not very impressive.",
    "I regret buying this phone, it’s terrible.",
]

# Initialize the CountVectorizer
vectorizer = CountVectorizer()

# Fit and transform the corpus to a document-term matrix
X = vectorizer.fit_transform(corpus)

# Convert the document-term matrix into a dense format (optional for visualization)
X_dense = X.toarray()

# Get the vocabulary (mapping of words to index positions)
vocab = vectorizer.get_feature_names_out()

# Print the vocabulary and document-term matrix
print("Vocabulary:", vocab)
print("Document-Term Matrix:\n", X_dense)
```

```
Vocabulary: ['but' 'buying' 'fine' 'impressive' 'is' 'it' 'like' 'not' 'perfectly'
 'phone' 'really' 'regret' 'terrible' 'the' 'this' 'very' 'works']
Document-Term Matrix:
 [[0 0 0 0 0 1 1 0 1 1 1 0 0 0 1 0 1]
 [1 0 1 1 1 0 0 1 0 1 0 0 0 1 0 1 0]
 [0 1 0 0 0 1 0 0 0 1 0 1 1 0 1 0 0]]
```

TF-IDF: An Extension to Bag of Words

Term Frequency (TF) represents the frequency of a term in a document. Inverse Document Frequency (IDF) reduces the impact of commonly occurring words across multiple documents. The TF-IDF score is calculated by multiplying the two metrics.

```python
from sklearn.feature_extraction.text import TfidfVectorizer
# Sample corpus
corpus = [
    "Python is easy to learn and enjoyable.",
    "Beyond being enjoyable, Python is extremely versatile.",
    "Practicing Python makes coding more exciting.!",
]

# Create the Tf-idf vectorizer
tfidf_vectorizer = TfidfVectorizer()

# Fit and transform the corpus
X_tfidf = tfidf_vectorizer.fit_transform(corpus)

# Show the Vocabulary
print("Vocabulary:", tfidf_vectorizer.get_feature_names_out())

# Show the TF-IDF Matrix
print("TF-IDF Representation:")
print(X_tfidf.toarray())
```

```
Vocabulary: ['and' 'being' 'beyond' 'coding' 'easy' 'enjoyable' 'exciting' 'extremely'
 'is' 'learn' 'makes' 'more' 'practicing' 'python' 'to' 'versatile']
TF-IDF Representation:
[[0.4261835  0.         0.         0.         0.4261835  0.32412354
  0.         0.         0.32412354 0.4261835  0.         0.
  0.         0.25171084 0.4261835  0.        ]
 [0.         0.4261835  0.4261835  0.         0.         0.32412354
  0.         0.4261835  0.32412354 0.         0.         0.
  0.         0.25171084 0.         0.4261835 ]
 [0.         0.         0.         0.43238509 0.         0.
  0.43238509 0.         0.         0.         0.43238509 0.43238509
  0.43238509 0.2553736  0.         0.        ]]
```

```python
!pip install nltk
import nltk
nltk.download('punkt')
nltk.download('punkt_tab')
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.3.1)
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.3)
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.1)
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
True
```

```python
!pip install gensim
from gensim.models import Word2Vec
from nltk.tokenize import word_tokenize

# Sample corpus
corpus = [
    "I visited the new cafe downtown today.",
    "The coffee there was rich and flavorful.",
    "I plan to go back again this weekend."
]

# Tokenize the corpus
tokenized_corpus = [word_tokenize(sentence.lower()) for sentence in corpus]

# Train the Word2Vec model
model = Word2Vec(sentences=tokenized_corpus, vector_size=100, window=5, min_count=1, sg=0)

# Get the vector representation of a word
word_vector = model.wv['cafe']
print(word_vector)
```

```
Requirement already satisfied: gensim in /usr/local/lib/python3.12/dist-packages (4.4.0)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.0.1)
[ 0.00973852 -0.00978219 -0.00650014  0.00278488  0.00642841 -0.00536813
  0.0027541   0.00912294 -0.00681663 -0.00609991 -0.00498817 -0.00367506
  0.00185016  0.00968344  0.00643634  0.00039487  0.00247003  0.00843968
  0.00912873  0.00562707  0.00594809 -0.00761902 -0.00382845 -0.00567823
  0.00618228 -0.00225825 -0.00877929  0.00762021  0.00839905 -0.00332323
  0.00911317 -0.00073757 -0.00362433 -0.00038514  0.0001929  -0.00350642
  0.00281382  0.00572725  0.0068673  -0.00890785 -0.00219733 -0.00547932
```

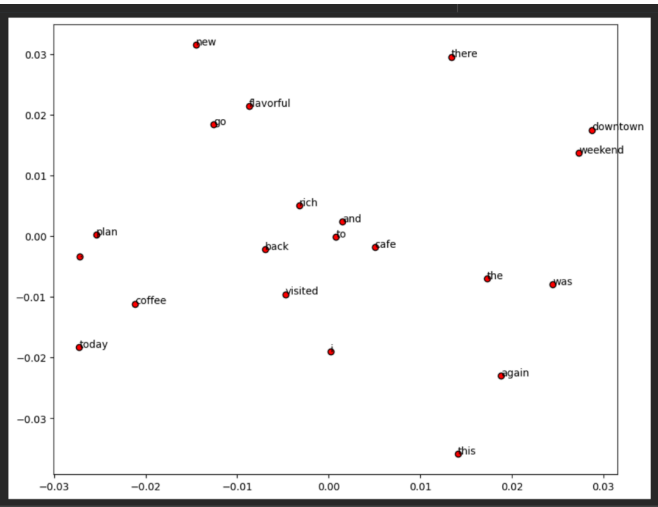
```
Requirement already satisfied: gensim in /usr/local/lib/python3.12/dist-packages (4.4.0)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.0.1)
[ 0.00973852 -0.00978219 -0.00650014  0.00278488  0.00642841 -0.00536813
  0.0027541   0.00912294 -0.00681663 -0.00609991 -0.00498817 -0.00367506
  0.00185016  0.00968344  0.00643634  0.00039487  0.00247003  0.00843968
  0.00912873  0.00562707  0.00594809 -0.00761902 -0.00382845 -0.00567823
  0.00618228 -0.00225825 -0.00877929  0.00762021  0.00839905 -0.00332323
  0.00911317 -0.00073757 -0.00362433 -0.00038514  0.0001929  -0.00350642
  0.00281382  0.00572725  0.0068673  -0.00890785 -0.00219733 -0.00547932
  0.00752192  0.00650132 -0.00436038  0.00232501 -0.00595765  0.00023932
  0.00946355 -0.00260823 -0.00519223 -0.00739442 -0.00291321 -0.00086506
  0.00353057  0.00973962 -0.00339072  0.00189861  0.00968124  0.00153217
  0.00098833  0.0098026   0.00929648  0.00770816 -0.0061724   0.00998423
  0.00584739  0.00907638 -0.00199794  0.00334917  0.00683652 -0.00388985
  0.00664177  0.00255917  0.00931593 -0.00303428 -0.00310603  0.00621641
 -0.00908108 -0.00725598 -0.00649853 -0.00074939 -0.00236312  0.00681956
  0.00923576 -0.00090886  0.00141448  0.00202182 -0.00202145 -0.00803223
  0.00744191 -0.00429788  0.00457729  0.00909261  0.00304661  0.0031397
  0.00405865 -0.00270265  0.00382775  0.00033813]
```

```python
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# Retrieve the word vectors and corresponding labels
words = list(model.wv.index_to_key)
word_vectors = [model.wv[word] for word in words]

# Reduce dimensions to 2D using PCA
pca = PCA(n_components=2)
word_vectors_2d = pca.fit_transform(word_vectors)

# Plot the words
plt.figure(figsize=(10, 8))
plt.scatter(word_vectors_2d[:, 0], word_vectors_2d[:, 1], edgecolors='k', c='r')
for i, word in enumerate(words):
    plt.annotate(word, (word_vectors_2d[i, 0], word_vectors_2d[i, 1]))
plt.show()
```



---

6. Result and Outcomes

Result

- The text documents were successfully preprocessed.
- A vocabulary containing all unique words was generated.
- Each document was converted into a **Bag of Words count vector**.

Outcomes

- Understood manual implementation of BoW without libraries
- Learned vocabulary construction and word frequency calculation
- Gained clarity on how text is transformed into numerical form
- Built a strong foundation for advanced NLP techniques

---

7. Conclusion

The Bag of Words model provides a simple yet effective way to represent text numerically. By implementing it from scratch, the internal working of text preprocessing, vocabulary creation, and vectorization becomes clear. Although it has limitations such as ignoring semantics and word order, BoW remains a fundamental technique in Natural Language Processing.